FML (CIS-452)
Assignment no. 2

Submitted by: Sara Nargis

Submitted to: Dr. Shahzad Ahmed Qureshi

Submitted on: May 11, 2026



Pakistan Institute of Engineering and Applied Sciences, Islamabad

1. Define the following performance measures with references (book, lecture, article, site address, etc.) for clustering.

2. For each measure, explain:

- *What it evaluates.*
- *The formula (if applicable).*
- *Its range and interpretation.*
- *Advantages and disadvantages.*

Clustering is an unsupervised learning technique, meaning we do not have "correct" labels to compare against. This makes evaluation different from supervised learning. Evaluation metrics for clustering fall into two main categories:

- Internal Evaluation: These metrics assess the quality of the clustering using only the data and the cluster assignments themselves, without requiring any ground truth labels. They evaluate properties like how compact the clusters are (points within a cluster are close together) and how well-separated the clusters are from each other.
- External Evaluation: These metrics compare the clustering results against a known ground truth (the true class labels). They are used when the true classes are known, allowing us to see how well the clustering algorithm discovered the natural groupings.

Silhouette Score

It is an internal evaluation metric. The Silhouette Score measures how similar a data point is to its own cluster compared to other clusters. It combines both the cohesion (how close points are within the same cluster) and separation (how far apart clusters are from each other) into a single score.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean distance to other points in the same cluster, and $b(i)$ is the mean distance to points in the nearest other cluster. The overall Silhouette Score is the average of $s(i)$ over all samples.

The score ranges from -1 to +1. A score close to +1 indicates that the sample is well-matched to its own cluster and poorly matched to neighboring clusters. A score near 0 means the sample lies on or very close to the decision boundary between two clusters. A negative score indicates that the sample might have been assigned to the wrong cluster. Advantages of this score are that it is simple, intuitive, and widely used. It provides a clear visual interpretation through silhouette plots. However, it is computationally expensive for large datasets. It assumes clusters are convex and spherical, making it less suitable for density-based or arbitrarily shaped clusters

References: https://developer.baidu.com/article/details/3218694

Davies-Bouldin Index

The Davies-Bouldin Index evaluates the average similarity between each cluster and its most similar one. It aims to identify sets of clusters that are compact (tightly grouped) and well-separated, like the Silhouette Score but approached differently.

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$

where σ_i is the average distance of points in cluster i to its centroid, $d(c_i, c_j)$ is the distance between centroids of clusters i and j, and k is the number of clusters.

Lower values of DB Index indicate better clustering. The minimum possible value is 0, and there is no fixed upper bound. A score of 0 indicates perfect separation. It does not require specifying the number of clusters and can be faster to compute than the Silhouette Score for large datasets. Disadvantages of DB Index include that it tends to favor convex clusters and can be sensitive to the distance metric used.

References: https://developer.baidu.com/article/details/3218694, weixin_43368424

Calinski-Harabasz Index

Also known as the Variance Ratio Criterion, this index measures the ratio of between-cluster dispersion to within-cluster dispersion. Higher values indicate that clusters are dense and well separated.

$$CH = \frac{\text{trace}(\mathbf{B})}{\text{trace}(\mathbf{W})} \times \frac{n - k}{k - 1}$$

where **B** is the between-cluster scatter matrix, **W** is the within-cluster scatter matrix, n is the total number of samples, and k is the number of clusters.

Higher scores indicate better clustering. There is no upper bound; the score can become arbitrarily large as clusters become more compact. CH is fast to compute, even for large datasets but like many internal indices, it assumes convex clusters and may not work well with density-based clusters.

References: https://developer.baidu.com/article/details/3218694

https://datascience.stackexchange.com/questions/78345/why-davies-bould-chose-a-number-ob-cluster-higher-than-silhouette-or-calinsky-ha/78448#78448

Adjusted Rand Index (ARI)

ARI measures the similarity between two clusterings (the predicted and the true labels), corrected for chance. It counts all pairs of samples that are assigned to the same or different clusters in both true and predicted clusterings. Based on the contingency table. The Rand Index is corrected for chance using the expected index of a random clustering, giving the Adjusted Rand Index. The score ranges from -1 (dissimilar) to 1 (perfect agreement). A value of 0 indicates that the clustering is no better than random assignment. Corrects for chance, making it a robust measure even when cluster sizes are imbalanced. It is symmetric and does not assume any cluster structure. Requires ground truth labels, which are not always available in unsupervised settings.

References:
https://www.sciencedirect.com/science/article/abs/pii/S0020025521010082

Normalized Mutual Information (NMI)

NMI quantifies the amount of information shared between the predicted and true clusterings. It normalizes the mutual information score so that it is bounded between 0 and 1.

$$NMI = \frac{2 \times I(Y;C)}{H(Y) + H(C)}$$

where $I(Y;C)$ is the mutual information between the true labels Y and predicted labels C, and $H(Y)$ and $H(C)$ are the entropies of Y and C.

NMI Ranges from 0 (no mutual information) to 1 (perfect correlation). Scores close to 1 indicate that the clustering perfectly recovers the true class structure.

Advantages: Independent of the absolute values of the labels and symmetric. It is also resilient to different cluster sizes.

Disadvantages: Requires ground truth labels and can be computationally intensive for large datasets.

Reference: https://scikit-learn.sourceforge.net/0.13/modules/generated/sklearn.metrics.normalized_mutual_info_score.html#sklearn.metrics.normalized_mutual_info_score

Purity

Purity measures the extent to which each cluster contains data points from a single class. It is a simple, intuitive measure of clustering quality.

$$\text{Purity} = \frac{1}{N} \sum_{i=1}^{k} \max_{j} |c_i \cap t_j|$$

where N is the total number of samples, c_i is the set of samples in cluster i, and t_j is the set of samples of class j. Ranges from 0 to 1, where 1 indicates a perfectly pure clustering (each cluster contains only data points from a single class). It is very simple to compute and interpret. It does not penalize having many clusters; a

clustering with as many clusters as samples would have a purity of 1. It also does not account for chance agreement.

Reference: https://hyperskill.org/learn/step/28809#internal-evaluation

Weighted Accuracy

Weighted Accuracy is the average of class-wise accuracies, weighted by the number of samples in each class. This is particularly useful when the dataset is imbalanced.

$$\text{Weighted Accuracy} = \sum_i \frac{\text{Number of samples in class i}}{\text{Total samples}} \times \text{Accuracy}_i$$

Ranges from 0 to 1, where higher is better. It gives more weight to classes with more samples. Provides a more realistic performance measure when classes are imbalanced compared to simple accuracy. It still requires ground truth labels and may not fully capture the clustering quality if cluster boundaries are not aligned with class boundaries.

References: https://blog.csdn.net/qq_44837312/article/details/130640321

Weighted F1-score

The F1-score is the harmonic mean of precision and recall. The weighted F1-score calculates the F1-score for each class individually, then averages them, weighting each by the number of samples in that class.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Weighted F1-score = $\sum_i w_i \times \text{F1}_i$, where w_i is the proportion of class i in the dataset.

Ranges from 0 to 1, where higher is better. The weighted version ensures that larger classes have a greater impact on the final score.

Advantages: Balances precision and recall, and the weighting handles class imbalance effectively.

Disadvantages: Requires ground truth labels and is not a direct measure of clustering quality but rather how well clusters correspond to classes.

References: https://blog.csdn.net/qq_44837312/article/details/130640321

Weighted F0.5-score

The F0.5-score is a variant of the Fβ-score that gives more weight to precision than recall. It is useful when false positives are considered more harmful than false negatives. The weighted version applies class-based weighting.

$$F_\beta = (1 + \beta^2) \times \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \times \text{Precision}) + \text{Recall}}$$

For $\beta = 0.5$, precision has twice the weight of recall. Weighted F0.5 uses the same weighting scheme as weighted accuracy and weighted F1.

Range & Interpretation: Ranges from 0 to 1, where higher is better. Values above 0.7 are generally considered good.

Advantages: Allows fine-tuning the balance between precision and recall according to the problem's needs. The weighted version still accounts for class imbalance.

Disadvantages: Requires ground truth labels and the choice of β is domain-specific.

References: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.fbeta_score.html

3. Choose a dataset for clustering analysis. You can select from standard datasets such as the Iris dataset, the Wine dataset, or any dataset from the UCI Machine Learning Repository. Preprocess the data as necessary (e.g., normalization, handling missing values).

The Iris dataset is an excellent choice here. It is a classic, small dataset (150 samples, 4 numeric features, 3 classes) that is easy to visualize and interpret. The natural groupings are known to be reasonably well separated, which makes it a perfect candidate for comparing clustering algorithms.

Because the features are measured in different units (e.g., petal length in cm, sepal width in cm), it is important to standardize the data to avoid features with larger scales dominating the distance calculations used by clustering algorithms. Standardization transforms the data to have zero mean and unit variance.

```
# Standardize features (zero mean, unit variance)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

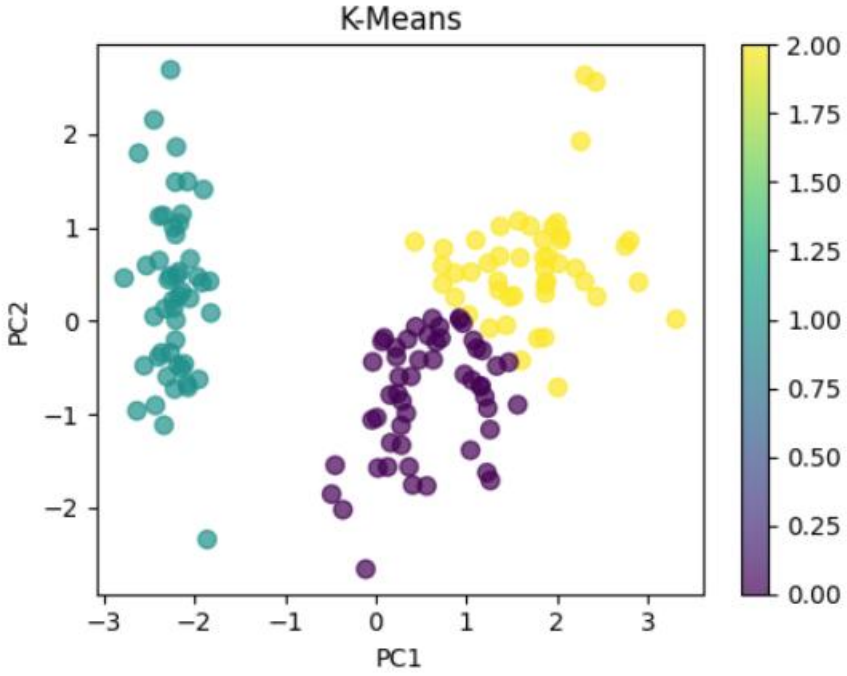
4. Implement clustering algorithms of K-Means, Agglomerative Clustering, and DBSCAN using a Python library such as scikit-learn. Apply these algorithms to your chosen dataset and obtain clustering results.

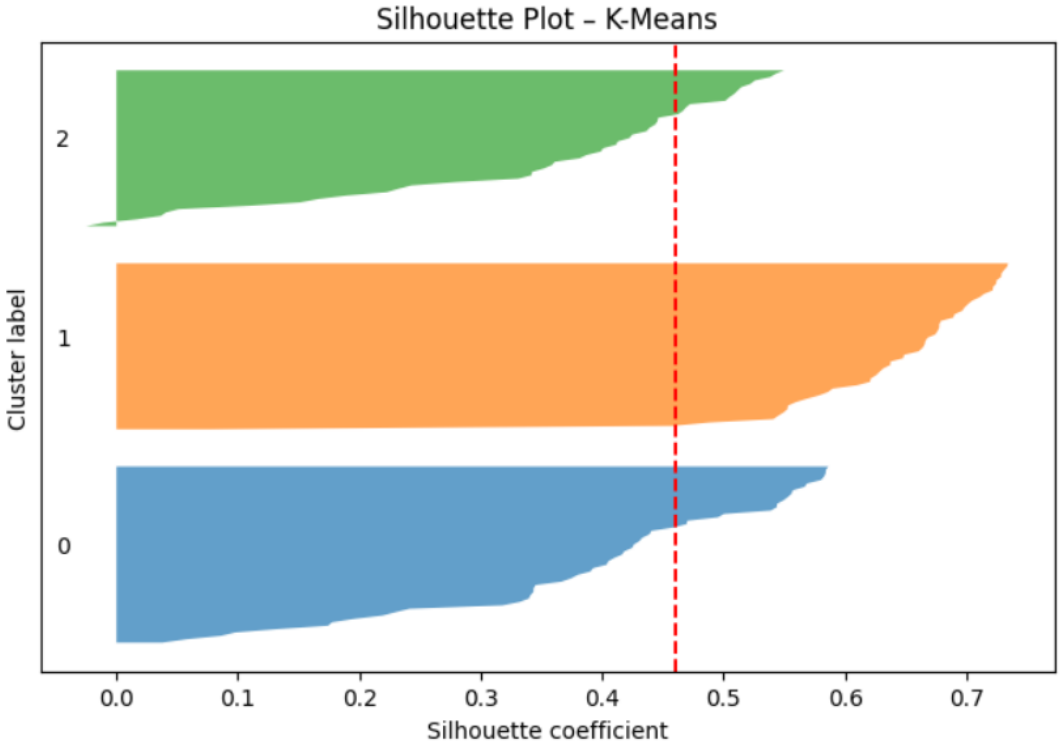
5. Implement the performance measures summarized in s. no. 1. If the measures are already available in libraries like scikit-learn, you can use those implementations but ensure you understand how they work.

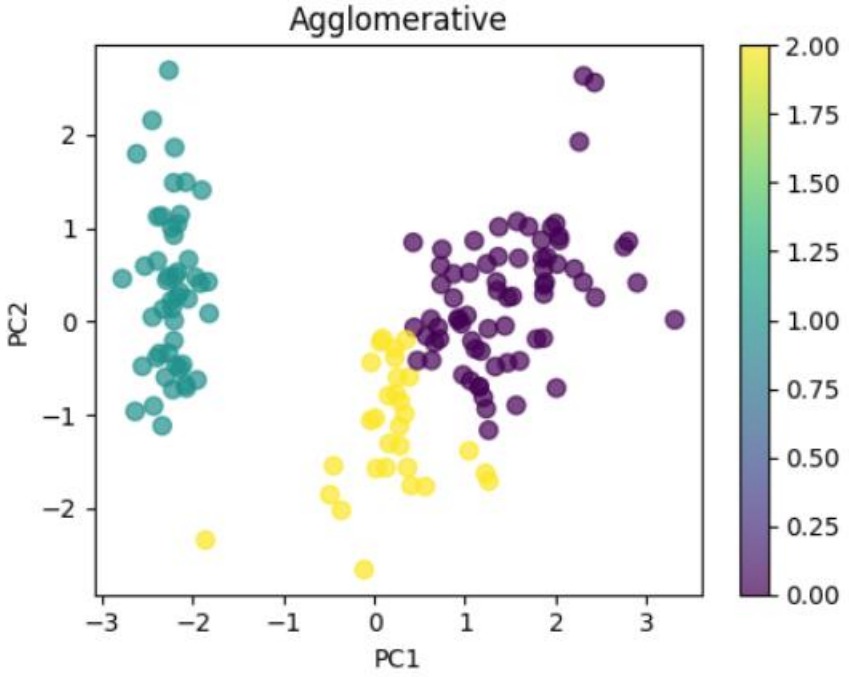
6. Evaluate the clustering results from s. no. 3 using the performance measures. Compare the performance of the different clustering algorithms based on these measures using an appropriate table.

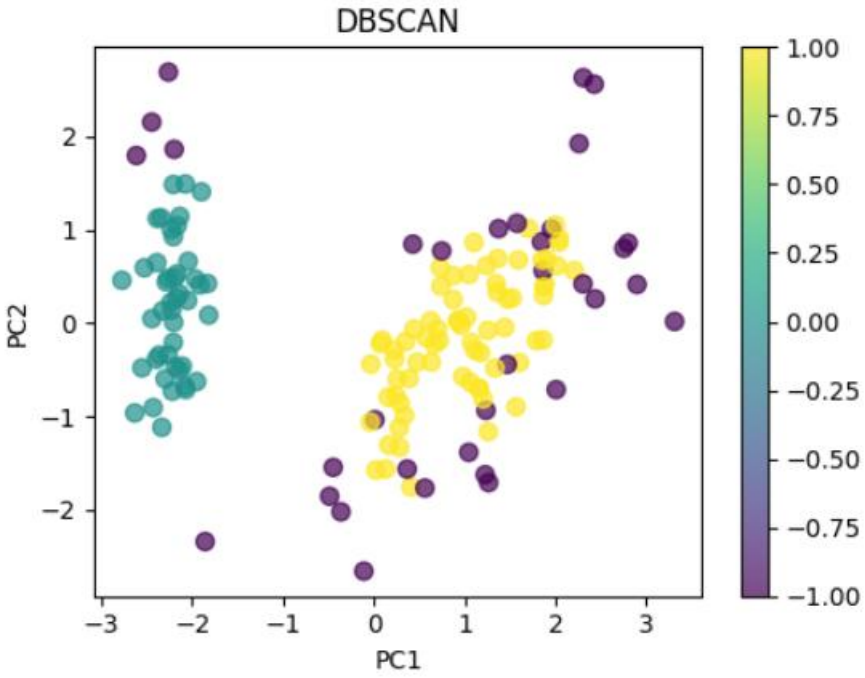
Implementation of clustering algorithms using scikit-learn and calculation of performance measures is shown in jupyter notebook.

Clustering results on Iris dataset (PCA reduced!)




Silhouette Plot – K-Means
Cluster label
2
1
0
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Silhouette coefficient


Agglomerative
PC2
2
1
0
−1
−2
−3
−2
−1
0
1
2
3
PC1
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00



Number of clusters found by DBSCAN: 2

Number of noise points (label -1): 34

Results analysis and comparison

Performance metrics comparison table:

Metric	K-Means	Agglomerative	DBSCAN
Silhouette Score ↑	0.4599	0.4467	0.3565
Davies-Bouldin Index ↓	0.8336	0.8035	7.1241
Calinski-Harabasz Index ↑	241.9	222.72	84.51
Adjusted Rand Index (ARI) ↑	0.6201	0.6153	0.4421
Normalized Mutual Info (NMI) ↑	0.6595	0.6755	0.5114
Purity ↑	0.8333	0.8267	0.6333
Weighted Accuracy ↑	0.8333	0.8267	0.68
Weighted F1-Score ↑	0.8332	0.8194	0.4733
Weighted F0.5-Score ↑	0.8335	0.8362	0.4223

In this table, ↑ means the smaller, the better and ↑ means the greater, the better.

Best performer: K-Means – It wins on 7 out of 9 metrics, often by a noticeable margin.

Second: Agglomerative – Very close to K-Means on most metrics, slightly better on Davies-Bouldin (lower is better) and NMI, and wins Weighted F0.5.

Worst: DBSCAN – It scores consistently lower on all metrics, with a disastrous Davies-Bouldin index (7.12 vs 0.8 for the others). Its weighted F1 is only 0.47, meaning it fails to correctly cluster a large portion of the data.

Why?

K-Means and Agglomerative (with Ward linkage) are both centroid-based / variance-minimizing algorithms. On a dataset like Iris, which has three relatively well-separated spherical clusters, they naturally perform similarly. That's why their scores are so close.

Conclusion

K-Means achieved the best overall performance across the majority of internal and external metrics, correctly recovering the three Iris species with approximately 83% accuracy. Agglomerative clustering performed nearly identically, while DBSCAN struggled due to its sensitivity to density parameters, producing many noise points and only two clusters. The results align with known properties of these algorithms: K-Means and Ward's hierarchical clustering favour compact, spherical clusters, whereas DBSCAN requires careful tuning to succeed on the Iris dataset.